

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
# =====
# Image Acquisition, Noise Simulation, Quantization, and Denoising Pipeline
# =====
# This notebook-ready script simulates the entire digital image pipeline:
# 1. Generates a continuous analog-like reference image.
# 2. Simulates sensor noise (Poisson Shot Noise & Gaussian Readout Noise).
# 3. Applies quantization (Bit-Depth mapping) showcasing detail loss.
# 4. Implements image restoration (Bilateral Filtering & Temporal Averaging).
# =====

import numpy as np
import matplotlib.pyplot as plt
import cv2 # OpenCV for advanced bilateral filtering

# --- 1. SETUP & SIMULATE ANALOG REFERENCE IMAGE ---
np.random.seed(42)
height, width = 512, 512

# Create a smooth, continuous intensity gradient representing the physical scene
x = np.linspace(0, 1, width)
y = np.linspace(0, 1, height)
X, Y = np.meshgrid(x, y)
# Base signal containing both smooth gradients and geometric structures
analog_scene = 180 * X + 50 * np.sin(Y * np.pi * 2) + 25
analog_scene = np.clip(analog_scene, 0, 255).astype(np.float32)
```

```

# --- 2. NOISE ACQUISITION SIMULATION ---
def add_sensor_noise(image, shot_noise_factor=1.5, readout_noise_std=8.0):
    """
    Simulates physical sensor noise:
    - Photon Shot Noise: Poisson-distributed, signal-dependent.
    - Readout Noise: Gaussian-distributed, signal-independent.
    """
    # Photon Shot Noise (Poisson processes scale with signal strength)
    noisy_poisson = np.random.poisson(image * shot_noise_factor) / shot_noise_factor

    # Readout / Thermal Noise (Zero-mean Gaussian noise)
    readout_noise = np.random.normal(0, readout_noise_std, image.shape)

    # Combined output signal
    noisy_image = noisy_poisson + readout_noise
    return np.clip(noisy_image, 0, 255).astype(np.float32)

noisy_sensor_image = add_sensor_noise(analog_scene)

# --- 3. QUANTIZATION FUNCTION ---
def quantize(image, bit_depth):
    """
    Converts a high-precision continuous signal into a discrete digital representation.
    """
    levels = 2 ** bit_depth
    # Normalize to [0, levels - 1], round to closest integer step, then scale back
    quantized = np.round(image * (levels - 1) / 255.0)
    reconstructed = quantized * (255.0 / (levels - 1))
    return reconstructed

# Quantize clean image to observe false contouring
quantized_3bit_clean = quantize(analog_scene, bit_depth=3)

# Quantize noisy image (8-bit Standard vs 3-bit Coarse)
quantized_8bit_noisy = quantize(noisy_sensor_image, bit_depth=8)
quantized_3bit_noisy = quantize(noisy_sensor_image, bit_depth=3)

# --- 4. NOISE MINIMIZATION (DENOISING) ---
# Method A: Spatial Denoising (Bilateral Filter to preserve edges while smoothing noise)
# We must convert to uint8 for OpenCV processing
noisy_8bit_uint8 = quantized_8bit_noisy.astype(np.uint8)
denoised_bilateral = cv2.bilateralFilter(noisy_8bit_uint8, d=9, sigmaColor=75, sigmaSpace=75)

# Method B: Temporal Averaging (Simulates capturing 16 frames of a static scene)
frames = [add_sensor_noise(analog_scene) for _ in range(16)]
temporal_average = np.mean(frames, axis=0)

# --- 5. VISUALIZATION AND ANALYSIS ---
fig, axes = plt.subplots(2, 3, figsize=(18, 11), dpi=100)
axes = axes.ravel()

# Plot 1: Clean Reference Scene
axes[0].imshow(analog_scene, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("1. Original 'Analog' Scene\n(Smooth Continuous Gradients)", fontsize=11, fontweight='bold')
axes[0].axis('off')

# Plot 2: Coarse 3-Bit Quantization (False Contouring)
axes[1].imshow(quantized_3bit_clean, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("2. 3-Bit Quantized Clean Image\n(Shows False Contouring/Posterization)", fontsize=11, fontweight='bold')
axes[1].axis('off')

# Plot 3: Acquired Noisy Image (Standard 8-Bit)
axes[2].imshow(quantized_8bit_noisy, cmap='gray', vmin=0, vmax=255)
axes[2].set_title("3. Acquired Noisy Sensor Image\n(Photon Shot + Readout Noise)", fontsize=11, fontweight='bold')
axes[2].axis('off')

# Plot 4: Quantized Noisy Image (3-Bit)
axes[3].imshow(quantized_3bit_noisy, cmap='gray', vmin=0, vmax=255)
axes[3].set_title("4. Noisy Image Quantized to 3-Bits\n(Noise Masked by Coarse Steps)", fontsize=11, fontweight='bold')
axes[3].axis('off')

# Plot 5: Spatial Denoising (Bilateral Filter)
axes[4].imshow(denoised_bilateral, cmap='gray', vmin=0, vmax=255)
axes[4].set_title("5. Restored Image (Bilateral Filter)\n(Smooths Noise, Preserves Gradients)", fontsize=11, fontweight='bold')
axes[4].axis('off')

# Plot 6: Temporal Averaging (16 Frames)
axes[5].imshow(temporal_average, cmap='gray', vmin=0, vmax=255)

```

```

axes[5].imshow(temporal_average, cmap=gray, vmin=0, vmax=255)
axes[5].set_title("6. Restored Image (Temporal Averaging)\n(Averaging 16 Frames to Cancel Noise)", fontsize=11, fontweight='bold')
axes[5].axis('off')

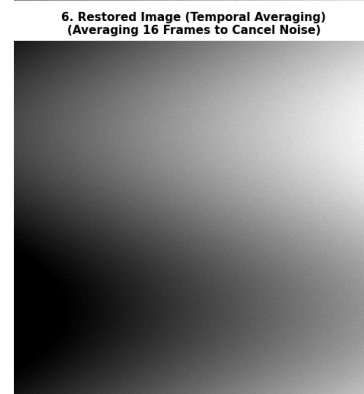
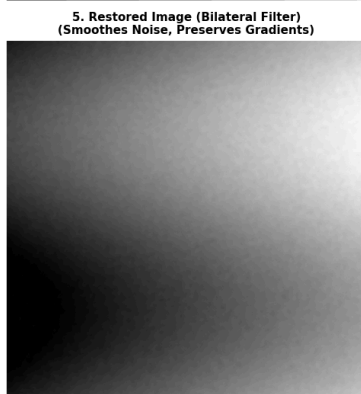
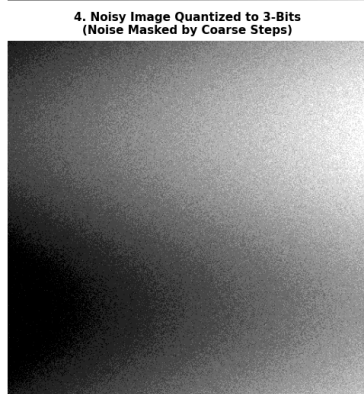
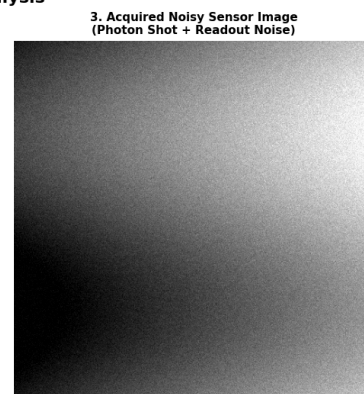
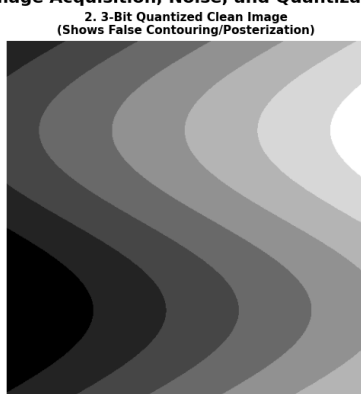
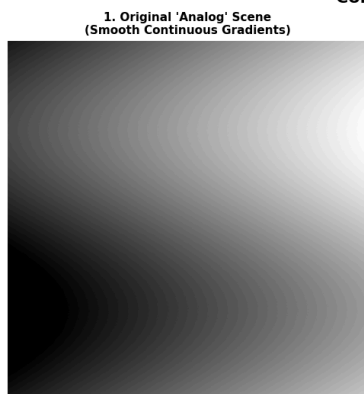
plt.suptitle("Complete Image Acquisition, Noise, and Quantization Analysis", fontsize=16, fontweight='bold', y=0.98)
plt.tight_layout()
plt.show()

# --- 6. METRIC COMPUTATION ---
mse_noisy = np.mean((analog_scene - quantized_8bit_noisy) ** 2)
mse_denoised = np.mean((analog_scene - denoised_bilateral) ** 2)
mse_temporal = np.mean((analog_scene - temporal_average) ** 2)

print("\n" + "="*50)
print("          QUANTITATIVE PERFORMANCE METRICS          ")
print("="*50)
print(f"Acquisition Noise Floor (MSE) : {mse_noisy:.4f}")
print(f"Post-Bilateral Filter (MSE)   : {mse_denoised:.4f}")
print(f"Post-Temporal Average (MSE)   : {mse_temporal:.4f}")
print("="*50)

```

Complete Image Acquisition, Noise, and Quantization Analysis



```

=====
          QUANTITATIVE PERFORMANCE METRICS
=====
Acquisition Noise Floor (MSE) : 137.9560
Post-Bilateral Filter (MSE)   : 3.5432
Post-Temporal Average (MSE)   : 9.0677
=====

```

Colab notebooks execute code on Google's cloud servers, meaning you can leverage the power of Google hardware, including [GPUs](#) and [TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU Runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.